Assignment #9

Conversational AI with Memory & Chains

Build an AI Chatbot Platform with Persistent Memory, LangChain Chains, Multi-Session Context & Knowledge Persistence

Organization:	Excellence Technologies Pvt Ltd
Phase:	Phase 2 - LangChain & Advanced RAG
Backend:	Flask (Python)
Frontend:	React.js
Duration:	3 - 4 Days
Difficulty:	Advanced

Memory & Chain Types Covered

Buffer Memory | Summary Memory | Entity Memory | Knowledge Graph Memory

Sequential Chains | Parallel Chains | Branching Chains | LCEL Pipelines

Session Persistence | Context Window Optimization | Long-Term Memory Store

This is an advanced assignment. Read all sections carefully before beginning.

1. Assignment Overview

In this assignment, you will build a Conversational AI Platform that focuses on the hardest problem in chatbot development: memory. Specifically, how an AI assistant remembers what you said earlier in the conversation, remembers facts about you across sessions, manages context when conversations get long, and uses different types of memory for different purposes.

You will implement every major memory type available in LangChain (buffer, summary, entity, knowledge graph), build chain architectures (sequential, parallel, branching), handle context window limitations gracefully, and create a persistent long-term memory store that survives across sessions. The platform will allow users to create multiple chatbot personas, each with their own memory configuration, and test how different memory strategies affect conversation quality.

What You Will Demonstrate

- Deep understanding of LangChain memory types and when to use each one
- Building LangChain chains: sequential (one output feeds the next), parallel (run simultaneously), branching (conditional routing)
- Context window management: when conversation exceeds token limit, intelligently compress/summarize history
- Persistent memory: conversations survive server restarts, users return and the AI remembers them
- Entity memory: the AI tracks and updates facts about people, places, and things mentioned in conversation
- Knowledge graph memory: building a structured graph of relationships mentioned in conversations
- Multi-session management: creating, switching, searching, and exporting conversation sessions
- Building a Flask backend with proper session handling, database persistence, and LangChain integration

Why Memory Matters

Without memory, every message is treated in isolation. An AI that forgets what you said 5 messages ago is useless for real work. But naive memory (send entire chat history) fails when conversations get long (token limits). This assignment teaches you to build memory systems that are both effective and efficient.

2. Problem Statement

Build a Conversational AI Platform where users can:

1. Chat with an AI assistant that remembers the full conversation context and intelligently manages memory as conversations grow long
2. Switch between different memory strategies (buffer, summary, entity, knowledge graph) per conversation and observe how each affects the AI's behavior
3. Create multiple chatbot personas, each with custom system prompts, personality, and memory configuration
4. Start a conversation, close the browser, come back days later, and the AI remembers everything from the previous sessions

5. View the AI's entity memory: a live dashboard showing what facts the AI has learned about entities (people, projects, companies) mentioned in conversation
6. View the AI's knowledge graph: a visual graph of relationships the AI has extracted (e.g., 'John works at Acme', 'Acme is a tech company')
7. Manage conversations: create, rename, search by content, pin important ones, export as Markdown/PDF, delete
8. Compare memory strategies: run the same conversation flow with different memory types and see which produces better responses

Example Scenarios

- **Long-Running Project Discussion:** User discusses a project over 50+ messages. With buffer memory, the AI forgets early details. With summary memory, it retains key points. With entity memory, it tracks all people, deadlines, and decisions. User can compare which memory type handles the long conversation best.
- **Personal Assistant:** User tells the AI their preferences over multiple sessions: 'I prefer Python over Java', 'My team uses React', 'Our deadline is March 15'. The AI stores these as entities and recalls them in future sessions without the user repeating themselves.
- **Knowledge Building:** As a user discusses their company structure, the AI builds a knowledge graph: 'CEO -> manages -> VP Engineering -> manages -> Team Lead -> reports to -> VP Engineering'. User can visualize this graph and query it.

3. Technical Requirements

3.1 Memory Types Implementation

Implement all four major memory types. Each must be selectable per conversation:

- **(a) Buffer Memory (ConversationBufferMemory):** Stores the entire conversation history verbatim. Simplest and most accurate but hits token limits on long conversations. Best for short conversations (< 20 messages). When context window limit is reached, truncate oldest messages.
- **(b) Summary Memory (ConversationSummaryMemory):** After every N messages (configurable, default 5), uses the LLM to summarize the conversation so far into a condensed paragraph. Replaces full history with: summary + last N recent messages. Token-efficient for long conversations. The summary should preserve key facts, decisions, and context.
- **(c) Entity Memory (ConversationEntityMemory):** Tracks entities (people, organizations, projects, dates, etc.) mentioned in conversation. Maintains an entity store: {'John': 'Software engineer at Acme, working on Project X, prefers Python', 'Project X': 'Due March 15, uses React frontend, 3 team members'}. Updates entity info as new facts emerge. Injects relevant entity context into each prompt.
- **(d) Knowledge Graph Memory (ConversationKGMemory):** Extracts subject-predicate-object triples from conversation and builds a graph. E.g., (John, works_at, Acme), (Acme, is_a, tech_company), (Project X, deadline, March 15). Queries the graph to inject relevant relationship context into prompts. Visualizable as an interactive graph.

Memory Type	Token Efficient	Fact Accuracy	Long Conversa	Best Use Case
Buffer	No (worst)	Perfect	Poor	Short, precise conversations
Summary	Good	Good (lossy)	Good	Long discussions, general context
Entity	Good	Very Good	Very Good	Fact-tracking, personal assistant
Knowledge Graph	Good	Very Good	Excellent	Relationship mapping, research

3.2 Hybrid Memory Strategy

Implement a hybrid memory that combines multiple types for best results:

- **Combined Memory:** Use Summary Memory for general context + Entity Memory for specific facts simultaneously. The prompt gets: conversation summary + relevant entity details + recent messages.
- **Auto-Switching:** (Good-to-Have) Start with Buffer memory. When conversation exceeds 15 messages, auto-switch to Summary + Entity hybrid. Notify the user of the switch.
- **Memory Configuration Panel:** Let users configure: which memory types are active, summary interval (every N messages), max buffer size, max entity store size, context window budget allocation (how many tokens for summary vs. entities vs. recent messages).

3.3 LangChain Chains Architecture

Build multiple chain types using LangChain LCEL:

- **Simple Conversation Chain:** Basic chat: system prompt + memory + user message -> LLM -> response. The default chain for general conversation.
- **Sequential Chain:** Multi-step processing. E.g., User asks a question -> Step 1: Classify intent (question/instruction/small-talk) -> Step 2: If question, enrich with entity context -> Step 3: Generate response -> Step 4: Extract and update entities. Output of each step feeds the next.
- **Parallel Chain:** Run multiple sub-chains simultaneously. E.g., When user sends a message, simultaneously: (1) Generate the response, (2) Extract entities, (3) Update knowledge graph, (4) Generate a conversation summary update. Merge results.
- **Branching/Router Chain:** Conditional routing based on message content. E.g., If user asks a factual recall question -> use entity-heavy prompt. If user asks for analysis -> use summary-heavy prompt. If small talk -> use minimal context. Route based on LLM classification of the message.
- **Refinement Chain:** Generate initial response -> Evaluate quality -> If the response does not reference known entities when it should, re-generate with explicit entity injection.

3.4 Context Window Management

Handle the fundamental LLM constraint: limited context window:

- **Token Budget System:** Define a total token budget (e.g., 4000 tokens for context). Allocate: 500 for system prompt, 1000 for memory (summary + entities), 1500 for recent messages, 1000 reserved for generation. Enforce these limits strictly.
- **Smart Truncation:** When recent messages exceed their budget, remove oldest messages first but NEVER remove the last user message or any message that contains entity definitions. Truncate from the middle of history, not the beginning (beginning often contains important setup context).
- **Summarization Trigger:** When buffer memory approaches 80% of its token budget, trigger an automatic summarization. Replace all messages except the last 5 with a summary paragraph.
- **Token Counter:** Show real-time token usage in the UI: 'Context: 2,847 / 4,000 tokens (71%)' with a visual progress bar. Warn when approaching limit.
- **Overflow Handling:** If even after summarization the context is too large, progressively compress: first summarize, then reduce entity detail, then keep only last 3 messages. Never fail silently - always show what was compressed.

3.5 Persistent Memory & Session Management

Conversations and memory must survive across browser sessions and server restarts:

- **Database Persistence:** Store everything in a database (SQLite/PostgreSQL): conversations, messages, entity stores, knowledge graph triples, summaries. Use SQLAlchemy ORM.

- **Session Management:** Users can: create new conversations (with name), switch between conversations (sidebar list), continue old conversations (all memory restored), rename/pin/archive conversations.
- **Memory Restoration:** When a user opens an old conversation, fully restore: message history, entity memory state, knowledge graph state, current summary. The AI should be able to continue exactly where it left off.
- **Cross-Session Entity Memory:** (Good-to-Have) Entity memory that persists across ALL conversations. Facts learned in Conversation A are available in Conversation B. E.g., user tells the AI their name in one conversation, AI remembers it in a new conversation.
- **Memory Export/Import:** Export a conversation's full memory state (messages + entities + graph + summary) as JSON. Import to restore or share.

3.6 Chatbot Personas

Allow users to create custom chatbot personas:

- **Persona Configuration:** Each persona has: name, avatar, system prompt, personality description, default memory strategy, temperature setting, and domain focus (general, technical, creative, business).
- **Pre-Built Personas:** Include at least 5 built-in personas: (1) General Assistant, (2) Code Helper (technical, precise, entity-tracks functions/files), (3) Creative Writer (imaginative, story-tracks characters), (4) Business Analyst (formal, entity-tracks metrics/KPIs), (5) Study Buddy (educational, knowledge-graph-builds concept maps).
- **Persona-Memory Pairing:** Each persona has a recommended memory strategy. Code Helper uses Entity Memory (tracks code artifacts). Creative Writer uses Knowledge Graph (tracks character relationships). Business Analyst uses Summary + Entity hybrid.
- **Persona Switching:** Switch persona within a conversation. Memory is preserved but the system prompt and behavior changes.

3.7 Entity & Knowledge Graph Visualization

Make the AI's internal knowledge visible and interactive:

- **Entity Dashboard:** Real-time display of all entities the AI is tracking. Table format: entity name, type (person/org/project/date), current description, last updated, source message. Updates live as conversation progresses.
- **Entity Timeline:** Show how an entity's description evolved over the conversation. E.g., 'Project X: v1: New project -> v2: Due March 15 -> v3: Due March 15, uses React, 3 members.'
- **Knowledge Graph Visualization:** Interactive graph rendered using D3.js, vis.js, or react-force-graph. Nodes = entities, edges = relationships. Click a node to see all known facts. Click an edge to see the source message.
- **Memory Search:** Search across all stored entities and knowledge graph triples. E.g., search 'deadline' returns all entities with deadline information across all conversations.

3.8 Memory Comparison Tool

Compare how different memory types handle the same conversation:

- **A/B Memory Test:** Replay a conversation through two different memory strategies simultaneously. Show side-by-side: how each memory type represents the conversation context, and how the AI responds differently with each.
- **Memory Efficiency Metrics:** Track per memory type: total tokens used, compression ratio (original vs. stored), entity recall accuracy (did the AI remember key facts when asked?), response quality (user ratings).
- **Conversation Replay:** Save a conversation's messages. Replay them through a different memory configuration to see how the AI would have responded differently.

3.9 Backend API (Flask)

Method	Endpoint	Description
POST	/api/conversations	Create new conversation (persona, memory type)
GET	/api/conversations	List conversations (search, filter, paginate)
GET	/api/conversations/{id}	Get conversation with messages & memory state
PUT	/api/conversations/{id}	Update conversation (rename, pin, archive)
DELETE	/api/conversations/{id}	Delete conversation and all its memory
POST	/api/conversations/{id}/message	Send message, get AI response (streaming)
GET	/api/conversations/{id}/entities	Get entity memory state
GET	/api/conversations/{id}/graph	Get knowledge graph triples
GET	/api/conversations/{id}/summary	Get current conversation summary
GET	/api/conversations/{id}/tokens	Get token usage breakdown

POST	/api/conversations/{id}/export	Export conversation + memory (JSON/MD/PDF)
POST	/api/conversations/import	Import conversation from JSON
GET/POST	/api/personas	List / create chatbot personas
GET/PUT/DEL	/api/personas/{id}	Manage persona
POST	/api/compare/memory	A/B compare memory types on same conversation
GET	/api/entities/search	Search entities across all conversations
GET	/api/stats	Memory stats (entities, graph size, token usage)
GET	/api/health	Health check

4. Recommended Tech Stack

Backend (Flask + LangChain)

- **Framework:** Flask with Flask-CORS, Flask-SQLAlchemy, Flask-SocketIO (for streaming)
- **LangChain Memory:** ConversationBufferMemory, ConversationSummaryMemory, ConversationEntityMemory, ConversationKGMemory
- **LangChain Chains:** LCEL (RunnableSequence, RunnableParallel, RunnableBranch, RunnablePassthrough)
- **LLM:** Google Gemini (free tier), Groq (free tier), or OpenAI GPT
- **Database:** SQLite with SQLAlchemy (conversations, messages, entities, graph triples)
- **Token Counting:** tiktoken for accurate token estimation
- **Knowledge Graph Storage:** SQLAlchemy tables for triples (subject, predicate, object) or NetworkX for in-memory graph
- **Streaming:** Server-Sent Events (SSE) via Flask or Flask-SocketIO

Frontend (React.js)

- **Framework:** React.js 18+ (Vite) with TypeScript
- **Styling:** Tailwind CSS + shadcn/ui
- **Graph Visualization:** react-force-graph (recommended) or vis.js/react-vis-network for knowledge graph
- **Chat UI:** Custom chat components or chatscope/chat-ui-kit-react
- **Markdown:** react-markdown for AI response rendering
- **State:** Zustand for global state (conversations, active persona, memory config)
- **Charts:** Recharts for memory stats and token usage visualization
- **Export:** jspdf + html2canvas for PDF export, or server-side export

> **LangChain Memory Classes**
> Some LangChain memory classes are being deprecated in favor of custom implementations using LCEL. Check the latest LangChain docs. If a memory class is deprecated, implement the equivalent behavior manually using LCEL + database storage. The concepts are the same even if the API has changed.

5. Expected Project Structure

```
conversational-ai-platform/
|
|-- backend/
|   |-- app.py                      # Flask entry point
|   |-- config.py                   # Settings & env vars
|   |-- routes/
|   |   |-- conversations.py        # Conversation CRUD
|   |   |-- messages.py             # Message send/receive + streaming
|   |   |-- personas.py             # Persona management
|   |   |-- memory.py               # Memory inspection & comparison
```

```
|   |   |-- export.py                # Export/import endpoints
|   |-- services/
|   |   |-- memory_manager.py        # Memory type factory & management
|   |   |-- buffer_memory.py         # Buffer memory implementation
|   |   |-- summary_memory.py        # Summary memory implementation
|   |   |-- entity_memory.py         # Entity extraction & storage
|   |   |-- kg_memory.py             # Knowledge graph memory
|   |   |-- hybrid_memory.py         # Combined memory strategies
|   |   |-- chain_builder.py         # LCEL chain construction
|   |   |-- context_manager.py       # Token budget & window management
|   |   |-- llm_client.py            # LLM integration
|   |-- models/
|   |   |-- database.py              # SQLAlchemy models
|   |-- data/
|   |   |-- personas.json            # Built-in persona definitions
|   |-- requirements.txt
|   |-- Dockerfile
|
|-- frontend/
|   |-- src/
|   |   |-- components/
|   |   |   |-- ChatWindow.tsx       # Main chat interface
|   |   |   |-- MessageBubble.tsx    # Chat message display
|   |   |   |-- ConversationList.tsx # Sidebar conversation list
|   |   |   |-- PersonaSelector.tsx  # Persona picker
|   |   |   |-- MemoryConfig.tsx     # Memory strategy configuration
|   |   |   |-- EntityDashboard.tsx  # Entity memory display
|   |   |   |-- KnowledgeGraph.tsx   # Interactive graph visualization
|   |   |   |-- TokenUsage.tsx       # Token budget display
|   |   |   |-- MemoryComparison.tsx # A/B memory comparison
|   |   |   |-- ExportPanel.tsx      # Export options
|   |-- package.json
|
|-- .env.example
|-- .gitignore
|-- README.md
```

6. UI Screens & Layout

Screen 1: Chat Interface (Main)

- Left sidebar: conversation list (grouped by date), search bar, 'New Chat' button, persona avatar badges
- Center: chat window with message bubbles (user right, AI left), markdown rendering, streaming text animation
- Bottom: message input with send button, persona name displayed, memory type badge
- Right sidebar (collapsible): memory inspector panel with tabs for Entity View, Knowledge Graph, Summary, Token Usage
- Top bar: active persona display, memory strategy selector dropdown, conversation settings

Screen 2: Entity Memory Dashboard

- Table of all tracked entities: name, type (person/org/project/etc.), current description, message count, last updated
- Click entity to expand: full description, timeline of how it evolved, source messages
- Search and filter entities by type, conversation, or keyword
- Live updates: when chatting, entities appear/update in real-time in this panel

Screen 3: Knowledge Graph Visualization

- Interactive force-directed graph: nodes are entities, edges are relationships
- Node size proportional to number of connections. Color-coded by entity type
- Click node: shows all facts and relationships for that entity
- Click edge: shows the relationship type and source message
- Zoom, pan, drag nodes to rearrange. Filter by relationship type or entity type
- Graph grows in real-time as conversation progresses

Screen 4: Persona Manager

- Grid of persona cards: avatar, name, description, recommended memory type, usage count
- Create new persona: form with name, system prompt (large text area), personality description, default memory, temperature, domain
- Edit existing personas. Delete custom personas (cannot delete built-in).
- 'Test Persona' button: sends a test message to preview persona behavior

Screen 5: Memory Comparison

- Select a past conversation to replay
- Choose two memory strategies to compare
- Side-by-side: replay conversation through both strategies showing different AI responses
- Metrics comparison: token usage, response quality, entity recall accuracy

- Verdict: which memory strategy produced better results for this conversation type

Screen 6: Token Usage & Memory Stats

- Real-time token budget bar: system prompt | summary | entities | recent messages | generation
- Historical token usage chart over conversation length
- Memory stats: total entities tracked, graph triples, summary length, compression ratio
- Per-conversation and aggregate views

7. Feature Breakdown & Priority

Must-Have (Core - Required)

1. Flask backend with LangChain integration, SQLAlchemy database, proper session management
2. Buffer Memory: full conversation history, truncation when limit reached
3. Summary Memory: auto-summarize every N messages, summary + recent messages in prompt
4. Entity Memory: extract entities from messages, maintain entity store, inject relevant entities into prompt
5. Knowledge Graph Memory: extract subject-predicate-object triples, store in database, query for context
6. Memory type selector per conversation (switch between buffer/summary/entity/kg)
7. Sequential chain: classify intent -> enrich context -> generate response -> update memory (LCEL)
8. Context window management: token budget system with real-time counter in UI
9. Persistent storage: conversations and memory state saved to database, restored on reload
10. Multiple conversations: create, list, switch, rename, delete with sidebar navigation
11. At least 5 chatbot personas with different memory configurations
12. Entity dashboard: table showing all tracked entities with descriptions
13. Knowledge graph visualization: interactive graph (react-force-graph or vis.js)
14. Streaming responses via SSE
15. React.js frontend with chat UI, sidebar, memory inspector panel

Good-to-Have (Intermediate)

1. Hybrid memory: Summary + Entity combined for best-of-both-worlds
2. Parallel chain: simultaneously generate response + extract entities + update graph + update summary
3. Branching chain: route to different prompts based on message intent classification
4. Entity timeline: show how entity descriptions evolved over conversation
5. Cross-session entity memory: facts from Conversation A available in Conversation B
6. Memory comparison tool: replay conversation with different memory types, compare results
7. Conversation export as Markdown, JSON, or PDF
8. Memory search: search across entities and knowledge graph triples
9. Conversation pinning and archiving
10. Auto-switch memory: start with buffer, auto-switch to summary+entity when conversation gets long

Bonus (Advanced)

1. Memory import/export: export full memory state as JSON, import into new conversation
2. Conversation replay through different memory strategy with side-by-side comparison
3. Token usage analytics: charts showing token consumption patterns over time
4. Persona marketplace: users can share/import custom personas

Confidential - For Internal Use Only

5. Smart summarization: summarize differently based on conversation domain (technical vs. creative)
6. Memory garbage collection: auto-remove stale entities not mentioned in last N messages
7. Docker + docker-compose deployment
8. Conversation branching: fork a conversation to explore different directions while preserving shared history

Confidential - For Internal Use Only

8. Evaluation Criteria

Criteria	Weight	What We Look For
Memory Implementation	30%	All 4 types working, entities accurate, KG extracts triples correctly
LangChain Chains	20%	LCEL chains used, sequential/parallel/branching demonstrated
Persistence & Sessions	15%	Memory survives restarts, sessions managed, context restored
UI/UX	15%	Clean chat UI, entity dashboard, knowledge graph visualization, responsive
Code Quality	10%	Clean Flask structure, modular memory services, proper DB models
Documentation	5%	README, memory type explanations, chain architecture diagram
Bonus Features	5%	Hybrid, cross-session, comparison, export, auto-switch

What Impresses Us

- Entity memory that accurately extracts and updates facts as conversation evolves
- Knowledge graph that builds meaningful relationships, not random triples
- Summary memory that preserves critical context while dramatically reducing token count
- Context window management that handles 100+ message conversations without breaking
- Persona behavior that genuinely changes with different system prompts and memory configs
- Knowledge graph visualization that updates in real-time as the user chats
- Memory comparison that demonstrates clear differences between strategies

Common Mistakes to Avoid

- Entity memory that extracts entities but never uses them in prompts (extraction without injection)
- Summary memory that summarizes but loses critical facts (summaries too brief)
- Knowledge graph with only generic triples like ('user', 'said', 'hello') - triples must be meaningful
- No token counting: hitting context limits silently and getting truncated/error responses
- Conversations that do not persist: memory lost on page refresh or server restart
- All personas behaving identically despite different system prompts
- Not using LCEL: using deprecated LangChain patterns instead of modern composable chains

9. Submission Guidelines

What to Submit

1. GitHub Repository - Full commit history. Include built-in persona definitions.
2. README.md - Architecture diagram showing chain flows, memory type explanations with examples, screenshots of chat UI + entity dashboard + knowledge graph, setup instructions.

3. .env.example - All required API keys and configuration.
4. Demo Video (Recommended) - 5-7 min: demonstrate each memory type in a conversation, show entity dashboard updating live, show knowledge graph growing, demonstrate persistence (refresh browser, continue chatting), compare two memory types.

Submission Deadline

Submit within 4 days of receiving this assignment.

10. Getting Started Guide

Day 1: Flask Backend, Database & Buffer/Summary Memory

- Initialize Flask project with SQLAlchemy, Flask-CORS
- Design database models: Conversation, Message, Entity, KGTriple, ConversationSummary
- Implement conversation CRUD endpoints
- Implement Buffer Memory: store messages, inject full history into prompt, handle truncation
- Implement Summary Memory: summarize every N messages, store summary, inject summary + recent messages
- Build token counter and context window budget system
- Build basic LCEL conversation chain with memory injection
- Test via API: create conversation, send messages, verify memory persists

Day 2: Entity Memory, Knowledge Graph & Chains

- Implement Entity Memory: extract entities from messages using LLM, store in database, inject into prompts
- Implement Knowledge Graph Memory: extract triples using LLM, store in database, query for context
- Build memory manager factory: select memory type per conversation
- Build sequential chain: classify -> enrich -> generate -> update memory
- Build parallel chain: generate + extract entities + update graph simultaneously
- Create 5 built-in personas with different memory configurations
- Implement persona selection and system prompt injection

Day 3: React.js Frontend

- Initialize React project with Tailwind + shadcn/ui
- Build chat interface: message bubbles, streaming text, input bar
- Build conversation sidebar: list, search, create new, switch, rename, delete
- Build memory inspector right panel: entity table, knowledge graph (react-force-graph), summary view, token counter
- Build persona selector with persona cards
- Build memory configuration panel (dropdown + settings)
- Implement SSE streaming for AI responses

Day 4: Visualization, Polish & Submit

- Polish knowledge graph visualization: colors, zoom, click interactions
- Build entity timeline view (evolution of entity descriptions)
- Implement conversation export (JSON, Markdown)
- Test all memory types: verify buffer, summary, entity, KG each work correctly

- Test persistence: create conversation, send messages, close browser, reopen, continue
- Test long conversations (30+ messages) with each memory type
- Write README with architecture diagram and memory type comparisons
- Record demo video and submit

11. Helpful Resources

LangChain Memory & Chains

- LangChain Memory: python.langchain.com/docs/how_to/#memory
- LCEL (Expression Language): python.langchain.com/docs/concepts/lcel
- RunnableSequence/Parallel/Branch: search 'LangChain LCEL RunnableParallel'
- ConversationEntityMemory: search 'LangChain entity memory tutorial'
- ConversationKGMemory: search 'LangChain knowledge graph memory'

Knowledge Graph Visualization

- react-force-graph: github.com/vasturiano/react-force-graph
- vis-network: visjs.github.io/vis-network/docs/network/
- D3.js force simulation: d3js.org

Flask & Frontend

- Flask: flask.palletsprojects.com
- Flask-SQLAlchemy: flask-sqlalchemy.palletsprojects.com
- React.js: react.dev
- shadcn/ui: ui.shadcn.com
- tiktoken: pypi.org/project/tiktoken/

12. Questions?

This assignment is about making AI conversations feel natural and intelligent over time. The test is simple: can you have a 50-message conversation with your chatbot and have it remember a fact from message #3 when asked about it at message #50? That is what production-grade memory looks like.

Good luck! Build the chatbot that actually remembers.

--- End of Assignment #9 ---